

 FULLSTACK AUTHENTICATION & ROW LEVEL SECURITY

Creación de Usuarios Segura, RLS & Google OAuth

Aprende la arquitectura real de autenticación en Next.js App Router, protegiendo los datos de cada cliente en Supabase PostgreSQL y habilitando Social Login en minutos.

♦ Bloque 1 (Core Auth)

Base de datos `profiles`, Trigger Postgres automático, RLS y Formulario con Zod.

Break (30 min)

Nivelación & Validación de usuarios creados.

♦ Bloque 2 (OAuth)

Google Cloud Console, credenciales web, botón "Continuar con Google" y callback PKCE.

El Puente de Autenticación: `auth.users` → `public.profiles`

Por qué nunca debes mezclar las credenciales protegidas con los datos de tu aplicación.

`auth.users` (Esquema Protegido)

Esquema interno gestionado por Supabase. Almacena contraseñas hasheadas (bcrypt), tokens de sesión y correos confirmados.

```
-- Inaccesible directamente desde el cliente
SELECT * FROM auth.users;
-- ⚠️ Protegido por roles del sistema
```

`public.profiles` (Tu Base de Datos)

Tu tabla pública vinculada por clave foránea (`id = auth.uid()`). Guarda roles, avatar, nombres y relaciones de negocio.

```
CREATE TABLE public.profiles (
  id UUID REFERENCES auth.users(id) PRIMARY KEY,
  email TEXT NOT NULL,
  full_name TEXT,
  role TEXT DEFAULT 'user'
);
```

 **La Magia del Trigger Automático:** Cada vez que un usuario se registra (por correo o Google), Postgres ejecuta una función interna que inserta automáticamente su registro en `public.profiles` sin depender del frontend.

Row Level Security (RLS) & Gestión de Claves

El principio de privilegio mínimo: Tu base de datos es el guardia de seguridad definitivo.

1 NEXT_PUBLIC_SUPABASE_ANON_KEY

Pública en el navegador. **Sólo es segura** porque RLS filtra cada SELECT, INSERT y UPDATE según el usuario logueado.

2 SUPABASE_SERVICE_ROLE_KEY

Clave maestra privada. Salta todas las políticas RLS. **NUNCA** debe tener prefijo NEXT_PUBLIC_ ni enviarse al cliente.

3 auth.uid() = id

PostgreSQL inyecta el ID del token JWT verificado en cada consulta SQL. Si intentas consultar datos ajenos, Postgres devuelve [].

Política RLS en Acción

```
-- 1. Habilitar seguridad por fila
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;

-- 2. Permitir que cada usuario lea SÓLO su fila
CREATE POLICY "Ver mi propio perfil"
  ON public.profiles FOR SELECT
  USING (auth.uid() = id);

-- 3. Permitir editar SÓLO su propio perfil
CREATE POLICY "Editar mi propio perfil"
  ON public.profiles FOR UPDATE
  USING (auth.uid() = id);
```

Formulario Seguro con Zod & Prevención de Enumeración

Validaciones estrictas y mensajes de error genéricos para blindar el acceso.

📋 Validación con Zod

```
const AuthSchema = z.object({
  email: z.string().email("Correo inválido"),
  password: z
    .string()
    .min(8, "Mínimo 8 caracteres")
    .regex(/[A-Z]/, "Al menos una mayúscula")
    .regex(/[0-9]/, "Al menos un número"),
  fullName: z.string().min(2).optional(),
});
```

🛡️ Prevención de Enumeración de Usuarios

❌ **Inseguro:** "El correo no existe en la base de datos"

✅ **Seguro:** "Credenciales inválidas. Verifica tu correo y contraseña"

🎯 Meta antes del Break:

Crear un usuario en tu app local y verificar que aparezca en `auth.users` y en `public.profiles` en Supabase.



Break & Nivelación de Entornos

Toma un café, recarga energías y valida tu checklist de mitad de sesión.

✓ Checklist Bloque 1:

1. Tabla `public.profiles` creada con RLS activado.
2. Trigger `on_auth_user_created` instalado en Postgres.
3. Registro exitoso de usuario con email y contraseña desde tu app en `localhost:3000`.

Google Cloud Console: Tarjeta, \$300 USD & Gratuidad

Tranquilidad financiera total: Por qué Google no te cobrará por usar OAuth 2.0.

🛡️ Verificación Anti-Spam

Google solicita tarjeta **únicamente para verificar tu identidad** y evitar creación masiva de bots. Es un estándar de seguridad internacional.

🎁 \$300 USD Gratis

Otorga \$300 USD por 90 días. **No cobra automáticamente** al vencer. Si sale modal de "Activar cuenta completa", haz clic en **Cancelar**.

✨ OAuth es 100% Gratis

El inicio de sesión social con Google (OAuth 2.0) **no consume saldo** ni tiene costo dentro de las cuotas estándar de desarrollo.

⚙️ Configuración en Google Cloud:

1. **OAuth Consent Screen:** Tipo Externo • Estado En prueba (Testing).
2. **Usuarios de prueba (Test users):** Agrega tu correo personal para poder loguearte en modo Testing.

Conexión Google Cloud → Supabase Dashboard

Configuración de Callback URLs y llaves maestras de OAuth.

1. Google Cloud (ID de Cliente OAuth)

- A** Orígenes autorizados de JavaScript
`http://localhost:3000`
- B** URIs de redireccionamiento autorizados
`https://<tu-project-ref>.supabase.co/auth/v1/callback`

2. Supabase (Authentication > Providers)

- 1** **Habilitar Google**
Activar switch *"Enable Sign in with Google"*.
- 2** **Pegar Client ID & Secret**
Pegar credenciales generadas por Google y dar clic en **Save**.



Prompt Maestro para Antigravity

Copia y pega este prompt en el chat de Antigravity para integrar el botón en 30 segundos.

 PROMPT INTEGRACIÓN GOOGLE OAUTH[Copiar y Pegar en Antigravity](#)

"Actúa como desarrollador Frontend en nuestro proyecto. Necesito que agregues la opción de inicio de sesión con Google en la pantalla de Login: 1. En el cliente de Supabase existente, añade la función para autenticar con Google mediante: ``supabase.auth.signInWithOAuth({ provider: 'google', options: { redirectTo: `${window.location.origin}/auth/callback` } })`` 2. Agrega un botón visualmente estilizado con el logo oficial de Google y el texto 'Continuar con Google' o 'Iniciar sesión con Google'. 3. Asegura el manejo de estados de carga (loading spinner) al hacer clic y la gestión limpia de posibles errores en la interfaz."

Troubleshooting: Errores Frecuentes en Vivo

Los 3 errores clásicos que le ocurren al 90% de los desarrolladores la primera vez.

● **redirect_uri_mismatch**

Causa: La URL en Google Cloud no es idéntica a la de Supabase.

Solución: Revisa que no tenga espacios, ni barra final /, y use `https://`.

● **access_denied / No verificada**

Causa: La app está en modo *Testing* y tu correo no está en la lista.

Solución: Añade tu email en *OAuth consent screen* → *Test users*.

● **Popup se cierra sin sesión**

Causa: Falta la ruta de intercambio PKCE `/auth/callback`.

Solución: Pídele a AntigraVity crear `app/auth/callback/route.ts` con `exchangeCodeForSession`.



¡Login Seguro & Google OAuth Funcionando!

Has construido una arquitectura de autenticación con estándares de producción.

 Próxima Clase (Clase 4): UI de Alto Nivel con MCP & Stitch

Conectaremos servidores MCP directamente en Antigravity para transformar nuestros prototipos en interfaces SaaS con acabados profesionales en tiempo récord.